

プラットフォームと制限事項

機能対応表 (プラットフォーム別)

プラットフォーム	Bluetooth MIDI	USB MIDI	ネットワ ーク MIDI (RTP- MIDI)	Nearby Connections MIDI	アプリ 間 MIDI	USB MIDI 2.0	ネットワ ーク MIDI 2.0 (UDP MIDI 2.0)	Scriptable Audio (オプ ション)
iOS	○	○	○	○	○	○	△ (試 験的)	○ *
Android	○	○	△ (試 験的)	○	○	○	△ (試 験的)	○ *
Universal Windows Platform	-	○	△ (試 験的)	-	○	△ (制限 あり)	△ (試 験的)	○ *
Standalone macOS / Unity Editor macOS	○	○	○	○	○	○	△ (試 験的)	○ *
Standalone Linux / Unity Editor Linux	○	○	△ (試 験的)	-	○	○	△ (試 験的)	○ *
Standalone Windows / Unity Editor Windows	-	○	△ (試 験的)	-	○	△ (制限 あり)	△ (試 験的)	○ *
WebGL	○	○	-	-	-	-	-	-

記号の説明:

- 対応
- △ 対応しているが試験的 / 制限あり
- 未対応

* Scriptable Audio 統合は **Unity 6000.3+**、スクリプト定義 `FEATURE_SCRIPTABLE_AUDIO`、WebGL 以外のビルドターゲットが必要です。詳細は [ビルド後の処理](#) を参照してください。

制限事項 / 要件

Android

- USB MIDI: API レベル 12 (Android 3.1) 以上。
- Bluetooth MIDI: API レベル 18 (Android 4.3) 以上。
- アプリ間 MIDI: API レベル 23 (Android 6.0) 以上。
- MIDI 2.0: API レベル 23 (Android 6.0) 以上。
- **Mono** でビルドする場合:
 - レイテンシ（遅延）の問題が発生する可能性があります。
 - `armeabi-v7a` のみをサポートする場合があります。
 - 推奨: **IL2CPP** を使用してください。
`Project Settings > Player > Configuration > Scripting Backend`
- Nearby Connections MIDI:
 - API レベル 28 (Android 9) 以上が必要。
 - かつ API レベル 33 (Android 13) 以上でコンパイルされている必要があります。

Bluetooth MIDI ペリフェラルモード (Android のみ)

BLE MIDI セントラル（デバイスのスキャンと接続）として動作するだけでなく、Android は BLE MIDI ペリフェラル（アプリを MIDI デバイスとして公開）としても動作できます。

注意点:

- これは Android 限定の機能です。
- API エントリーポイントについては [MIDI 1.0 \(MidiManager\)](#) を参照してください。
- プラットフォームの権限と Bluetooth の状態が適用されます（[ビルド設定](#) を参照）。

Meta Quest (Oculus Quest)

Meta Quest デバイスは Android で動作するため、上記の **Android** の要件が適用されます。

実用上の注意点:

- Quest での **USB MIDI** は、Android ビルド後の処理で USB デバイスインテントフィルタを有効にする必要がある場合があります。
 - 参照: [ビルド後の処理とスクリプト定義シンボル](#) (Quest USB MIDI のセクション)
- **Bluetooth MIDI** デバイスの検索とペアリングには、Android の Companion Device ワークフローを使用できます。
 - スクリプト定義シンボルを有効化: `FEATURE_ANDROID_COMPANION_DEVICE`
 - Unity 6 以降で Application Entry Point に **GameActivity** を使う場合、ビルド後処理は `BleMidiUnityGamePlayerActivity` を選択します（[ビルド後の処理](#) 参照）

- 参照: [ビルド後の処理とスクリプト定義シンボル](#)

iOS / macOS

- サポートされている iOS: 12.0 以上。
- Bluetooth MIDI: セントラルモードのみ。

UWP

- サポートされている UWP ターゲットバージョン (MIDI 1.0) : 10.0.10240.0 以上。
- MIDI 2.0 (USB): **制限付き対応** (`UwpMidi2Plugin` / `Midi2Plugin.Uwp.cs`)。 `InitializeMidi2` 経由。
 - 実行要件:
 - OS: **Windows 11 24H2 以降** (ビルド `**26100+**`。25H2 / 26H1 含む)。Windows MIDI Services が有効であること
 - 端末に **Windows MIDI Services SDK Runtime and Tools** を別途インストール ([get-latest](#) または `winget install Microsoft.Windows.MIDIServicesSDK`)
 - アーキテクチャ: **x64** または **ARM64** (x86 非対応)
 - 参照は `Assets/MIDI/Plugins/WSA/Microsoft.Windows.Devices.Midi2.winmd` (ネイティブ WinRT。NetProjection は使用しない)
 - **UWP プレイヤービルドのみ** (`UNITY_WSA && !UNITY_EDITOR`)。Unity Editor / Standalone Windows では動作しない
 - Windows 10 および Windows 11 23H2 以前、または MIDI Services / SDK Runtime 未導入の端末では初期化をスキップし、警告ログを出して完了コールバックを呼ぶ (クラッシュしない)
 - MIDI 1.0 (`WindowsMidiPlugin`) と MIDI 2.0 (`UwpMidi2Plugin`) は別バックエンドのため、同一物理デバイスが **両方の API で列挙されることがある**。デバイス ID 空間も異なる (クラシック MIDI1 ID と Windows MIDI Services の `EndpointDeviceId`)。アプリ側で用途に応じて MIDI1 のみ / MIDI2 のみを選ぶこと
 - ホットプラグ (抜挿) は Watcher で処理する。UWP Suspend / `OnApplicationPause` 向けの専用再初期化は未実装 (長時間サスペンド後は `TerminateMidi2` → `InitializeMidi2` の再実行を推奨)
 - UWP ビルドでは `UdpMidi2Plugin` (試験的) も同時に初期化される
- Bluetooth MIDI はサポートされていません。
- RTP-MIDI を使用するには、以下の Capability を有効にする必要があります:
 - `Project Settings > Player > Capabilities > PrivateNetworkClientServer`

Windows (Standalone / Unity Editor)

- Bluetooth MIDI はサポートされていません。
- MIDI 2.0 (USB): **制限付き対応** (`WindowsMidi2Plugin` / `Midi2Plugin.Windows.cs`)。
`InitializeMidi2` とネイティブ `Midi2Native.dll` (方法 C — C++/WinRT) 経由。
 - 実行要件:

- OS: **Windows 11 24H2 以降**（ビルド **26100+**. 25H2 / 26H1 含む）。Windows MIDI Services が有効であること
- 端末に **Windows MIDI Services SDK Runtime and Tools** を別途インストール（[get-latest](#) または `winget install Microsoft.WindowsMIDIServicesSDK`）
- アーキテクチャ: **x64** または **ARM64**（x86 非対応）。Unity Editor Play Mode は **x64** DLL を使用
- ネイティブ: `Assets/MIDI/Plugins/Windows/x86_64/Midi2Native.dll` および `.../ARM64/Midi2Native.dll`。マネージ `Assets` に `WinRT.Runtime` / `NetProjection` / `Standalone` 用 `Midi2.winmd` は載せない
- **Standalone Windows および Unity Editor Windows**（`UNITY_EDITOR_WIN || (UNITY_STANDALONE_WIN && !UNITY_EDITOR)`）。UWP の USB MIDI 2.0 は **方法 B**（`UwpMidi2Plugin` + WSA `winmd`）のまま別バックエンド。Standalone 向けに WSA `winmd` を有効化しないこと
 - Windows 10 および Windows 11 23H2 以前、または MIDI Services / SDK Runtime 未導入の端末では初期化をスキップし、警告ログを出して完了コールバックを呼ぶ（クラッシュしない）
 - MIDI 1.0（`WindowsMidiPlugin` / `WinMM`）と MIDI 2.0（`WindowsMidi2Plugin`）は別バックエンドのため、同一物理デバイスが **両方の API で列挙される**ことがある。デバイス ID 空間も異なる（`WinMM ID` と `Windows MIDI Services` の `EndpointDeviceId`）。アプリ側で用途に応じて MIDI1 のみ / MIDI2 のみを選ぶこと
 - ホットプラグ（抜挿）は `Watcher` で処理する。Editor Play Mode 終了時は `PlayModeStateChanged` から `TerminateMidi`（ネイティブ SDK シャットダウンは `refcount` で安全）
 - Standalone / Editor で `InitializeMidi2` を使うと、USB MIDI 2.0 に加えて `UdpMidi2Plugin`（試験的）も初期化される

Linux

- 1 つの USB デバイスに MIDI 1.0 と MIDI 2.0 が共存している場合、MIDI 2.0 ポートのみが検出されることがあります。

WebGL

- デバイスのサポート状況は OS/ブラウザの WebMIDI 環境に依存します。
- WebGL は生の UDP/TCP ソケットを使用できないため、RTP-MIDI / UDP MIDI 2.0 は利用できません。
- WebGL では、他のサーバーへの `UnityWebRequest` アクセスが制限される場合があります。`.mid` ファイルなどのコンテンツには `StreamingAssets` を使用してください。
- WebGL は USB MIDI 2.0 デバイスやネットワーク MIDI 2.0 を処理できません。MIDI 2.0 ランタイム機能は、クリップファイルの読み書きを除いて利用できません。
- WebGL テンプレートで `unityInstance` を公開する必要がある場合があります（[トランスポートとプラットフォームの注意点](#)を参照）。
- **Scriptable Audio 統合は WebGL ではコンパイルされません**（`asmdef` で WebGL プラットフォーム除外）。`ScriptableAudioUtility.IsAvailable` は `false` です。

- **Maestro / MPTK (WebGL・オプション)** : Maestro 2.16 以降は公式 WebGL 対応を記載。**Core player** はレガシー (WebGL では `MPTK_Core` がオフ) 。コンテンツは `StreamingAssets` 、または `MidiExternalPlayer` / リモート SoundFont 向けに **CORS 準拠** のホストを推奨。WebGL の `MPTKWriter` はダイレクト再生寄り。**MPTK が使える ≠ プラグインの全統合が使える** (Scriptable Audio / Network MIDI / MIDI 2.0 ランタイムは上記どおり制限) 。詳細は [Maestro / MPTK 統合](#) 。

Scriptable Audio Pipeline（オプション統合）

要件	値
Unity バージョン	6000.3 LTS 以降
スクリプト定義	<code>FEATURE_SCRIPTABLE_AUDIO</code>
パッケージ	<code>com.unity.burst</code> 、 <code>com.unity.collections</code> （本リポジトリ利用時は同梱済み）
WebGL	非対応 — 統合 <code>asmdef</code> 除外、 <code>#if !UNITY_WEBGL</code> ガード
シンボル off / Unity 6.2 以前	統合コード非コンパイル、コア MIDI プラグインは変更なし

要件を満たさない場合でもプロジェクトはエラーなくコンパイルできる想定です。有効化手順は [ビルド後の処理 — Scriptable Audio Pipeline 統合](#) を参照してください。

Chunity / Chunity Scriptable Generator（オプション統合）

項目	内容
Chunity ランタイム	非同梱 。利用者が別途インストール
スクリプト定義	<code>FEATURE_CHUNITY</code> （必須）。Generator 利用時は追加で <code>FEATURE_CHUNITY_SCRIPTABLE_AUDIO</code>
パッケージ（Generator）	<code>com.unity.collections</code> （ <code>asmdef</code> が <code>Unity.Collections</code> を参照）
Generator パッチ	Chunity 本体への <code>useBuiltInAudioFilter</code> 最小パッチ（ Chunity Optional パッチ手順 ）
WebGL	Chunity 自体の制約に従う。Generator は非対応で FilterRead / WebChuckK ヘフオールバック
Unity バージョン（Generator）	6000.3+ を想定（本体 Scriptable Audio 統合と同系統）

詳細は [Unity エコシステム統合 — Chunity](#) および [ビルド後の処理](#) を参照してください。